



**UNIVERSIDAD PRIVADA DE TACNA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA DE SISTEMAS**



**Guía Práctica de Laboratorio
“Auditoría en Docker y custodia de la evidencia digital”**

**CURSO:
“AUDITORÍA DE SISTEMAS”
Laboratorio N.º 04**

AUTOR(ES):
Mamani Valdivia, Elvis · 2020068763 · A

Docente:
Dr. Oscar Juan Jimenez Flores

TACNA — PERÚ



Índice

Índice.....	1
Cómo se usa esta plantilla.....	3
1. Información sobre el evento práctico	4
1.1 Título del evento práctico	4
1.2 Objetivos	4
1.3 Tiempo de duración	4
1.4 Resultados de aprendizaje	4
1.5 Recursos.....	5
1.6 Seguridad.....	5
2. Procedimiento o metodología	5
Paso A	¡Error! Marcador no definido.
Paso B	¡Error! Marcador no definido.
Paso C	¡Error! Marcador no definido.
3. Resultados	6
4. Conclusiones	13
5. Cuestionario	13
6. Referencias bibliográficas	13
7. Anexos	13



Cómo se usa esta plantilla

Este es el formato en el que se entrega **todo informe de taller de SI-084**. Se llena en Word y se entrega en **PDF**.

1. Guarda una copia con el nombre **SI084-S<NN>-TALLER-Grupo<N>.pdf**. Por ejemplo, SI084-S03-TALLER-Grupo4.pdf.
2. Completa la carátula. Escribe los integrantes con apellidos, nombres y código, uno por línea.
3. Copia el enunciado de cada sección desde el archivo 3-TALLER.md de la semana. Las siete secciones son las del formato EPIS y van en este orden.
4. Actualiza el índice antes de exportar. Clic derecho sobre el índice y luego **Actualizar campos**.
5. Exporta con **Archivo** → **Guardar como** → **PDF**. No entregues el .docx.

Borra esta página antes de entregar.

1. Información sobre el evento práctico

1.1 Título del evento práctico

Evaluación automatizada de controles generales de TI mediante herramientas libres de auditoría de configuración (*compliance scanning*), contrastando la evidencia técnica contra los CIS Benchmarks y el Anexo A de la ISO/IEC 27001:2022.

1.2 Objetivos

- Ejecutar **Lynis** para auditar el endurecimiento del sistema operativo anfitrión.
- Ejecutar **OpenSCAP** con la guía SCAP Security Guide para evaluar cumplimiento contra un perfil normativo formal.
- Ejecutar **Docker Bench for Security** para auditar el plano de contenedores contra el CIS Docker Benchmark.
- Ejecutar **Trivy** para auditar vulnerabilidades de imágenes, malas configuraciones IaC y **secretos embebidos**.
- **Consolidar los cuatro reportes en una matriz de control única** mapeada a ISO/IEC 27001:2022 y COBIT 2019.
- Distinguir, sobre evidencia real, una **deficiencia de diseño** de una **deficiencia de eficacia operativa**.

1.3 Tiempo de duración

100 minutos.

1.4 Resultados de aprendizaje

- **RA1** Analiza e interpreta los conceptos y terminología de Auditoría de Sistemas.
- **RA2** Evalúa la seguridad de la información en Auditoría de Sistemas.



1.5 Recursos

Recurso	Detalle
Entorno si084-lab	Semanas 01–03 operativas
Lynis	https://cisofy.com/lynis/ — <code>docker run --rm -it --pid host --net host -v /:/rootfs:ro ...</code>
OpenSCAP + SCAP Security Guide	https://www.open-scap.org/ y https://github.com/ComplianceAsCode/content
Docker Bench for Security	https://github.com/docker/docker-bench-security
Trivy	https://trivy.dev/ — imagen aquasec/trivy
CIS Benchmarks	https://www.cisecurity.org/cis-benchmarks (descarga gratuita previo registro)
Python 3.11+ con pandas	Consolidación de reportes

1.6 Seguridad

1. Lynis y Docker Bench requieren acceso de lectura al sistema anfitrión; se ejecutan **solo en el equipo del propio estudiante**, nunca en un equipo compartido del laboratorio sin autorización del administrador.
2. Los reportes contienen el inventario de software y versiones del equipo. Se clasifican como **Confidencial** y **no se publican en repositorios públicos**.
3. Trivy en modo secret puede detectar credenciales reales presentes en el equipo. Si aparece un secreto real, **se reporta al docente y se elimina del reporte antes de entregarlo**; no se transcribe en el informe.
4. Ninguna herramienta se ejecuta contra equipos de la red del campus.

2. Procedimiento o metodología

Documento del caso para esta semana. La organización entrega **Narrativa del proceso de compras y pagos**, en CASOS/EMPRESA-<NN>-<slug>/documentos/narrativa-proceso-compras.md. Es consistente con los datos de datos/. Las personas, usuarios y proveedores que menciona existen en los archivos. **No señala sus debilidades**; declara lo que la organización dice hacer.

Paso A — Auditoría del sistema anfitrión con Lynis

```
mkdir -p 20_evidencia/E04_config && cd 20_evidencia/E04_config

docker run --rm -it \
  --pid host --net host \
  -v /:/rootfs:ro \
  -v "$PWD":/salida \
  docker.io/cisofy/lynis:latest audit system --forensics \
  --report-file /salida/lynis-report.dat | tee lynis-consola.txt

# Índice de endurecimiento (métrica que se sigue entre auditorías)
grep -E '^hardening_index=' lynis-report.dat
# Sugerencias y advertencias
grep -E '^(warning|suggestion)\[\\]=' lynis-report.dat | head -40
```

Lectura de auditor. El `hardening_index` es un número entre 0 y 100. Su valor absoluto importa menos que **su trayectoria entre auditorías** y **qué controles específicos lo deprimen**. Se identifican las tres advertencias de mayor severidad y se anota, para cada una, el control ISO correspondiente.

Paso B — Cumplimiento formal con OpenSCAP

Lynis da recomendaciones; OpenSCAP evalúa contra un **perfil normativo formal y versionado**, que es lo que un auditor puede citar como criterio.

```
docker run --rm -it -v "$PWD":/salida ubuntu:22.04 bash -c '
  apt-get update -qq && apt-get install -y -qq libopenscap8 ssg-debderived >/dev/null
  # Listar perfiles disponibles en la guía
  oscap info /usr/share/xml/scap/ssg/content/ssg-ubuntu2204-ds.xml | head -40
  # Evaluar contra el perfil CIS Level 1 Server
  oscap xccdf eval \
    --profile xccdf_org.ssgproject.content_profile_cis_level1_server \
    --results /salida/oscap-resultados.xml \
    --report /salida/oscap-reporte.html \
    /usr/share/xml/scap/ssg/content/ssg-ubuntu2204-ds.xml || true
'
```

Se abre `oscap-reporte.html` en el navegador. Para el papel de trabajo se extraen:

- Total de reglas evaluadas, aprobadas y fallidas.
- Las **cinco reglas fallidas de severidad alta**, con su identificador XCCDF completo (que es el criterio citable).



- Para cada una, la referencia cruzada al control ISO/IEC 27001 o NIST que la propia guía declara.

Paso C — CIS Docker Benchmark con Docker Bench

```
cd ../../ && git clone --depth 1 https://github.com/docker/docker-bench-security.git
cd docker-bench-security
sudo sh docker-bench-security.sh -l ../20_evidencia/E04_config/docker-bench.log
cd ../20_evidencia/E04_config
```

```
grep -E '^\[WARN\]' docker-bench.log | tee docker-bench-warn.txt | wc -l
grep -E '^\[INFO\]' docker-bench.log | head -20
```

Las secciones que más hallazgos producen y su lectura de auditoría:

Sección CIS Docker	Qué evalúa	Control ISO/IEC 27001:2022
1 — Configuración del anfitrión	Partición separada, versión del <i>daemon</i> , auditoría de archivos de Docker	A.8.9, A.8.15
2 — <i>Daemon</i> de Docker	TLS, registros de confianza, <i>users-remap</i> , tráfico entre contenedores	A.8.20, A.8.21
4 — Imágenes	Usuario no root, imágenes firmadas, HEALTHCHECK, ausencia de secretos	A.8.28, A.8.31
5 — <i>Runtime</i>	Privilegios, capacidades, montaje del socket de Docker, límites de recursos	A.8.9, A.8.6

Hallazgo casi garantizado. La regla 5.31 («el socket de Docker no debe montarse dentro de contenedores») y la 4.1 («los contenedores deben ejecutarse con un usuario distinto de root») fallan en la mayoría de laboratorios. Ambas se traducen en una **escalada de privilegios del contenedor al anfitrión**, y esa es la redacción de negocio que debe llevar el hallazgo.

Paso D — Vulnerabilidades, laC y secretos con Trivy

```
TRIVY="docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v $PWD:/out aquasec/trivy"
```

```
# 1. Vulnerabilidades de las imágenes del entorno auditado
for IMG in bkimminich/juice-shop:latest postgres:16 wordpress:latest mariadb:11; do
  N=$(echo $IMG | tr '[:/]' '___')
  $TRIVY image --severity HIGH,CRITICAL --format json -o /out/trivy_${N}.json $IMG
  $TRIVY image --severity HIGH,CRITICAL $IMG | tee -a /out/trivy_resumen.txt
done
```

```
# 2. Malas configuraciones en la infraestructura como código
$TRIVY config --severity HIGH,CRITICAL /out/../../entorno | tee /out/trivy_iac.txt

# 3. Secretos embebidos en el repositorio de trabajo
$TRIVY fs --scanners secret /out/../../ | tee /out/trivy_secretos.txt

# 4. Inventario de componentes (SBOM) – exigido crecientemente en contratos
$TRIVY image --format cyclonedx -o /out/sbom_juiceshop.json bkimminich/juice-shop:latest
```

Por qué importa el SBOM. El *Software Bill of Materials* es el inventario de componentes de un producto de software. Sin él, ante la publicación de una vulnerabilidad crítica en una biblioteca, la organización no puede responder en horas la pregunta «¿estamos afectados?». Ese es un **control detectivo de gestión de vulnerabilidades** (ISO/IEC 27001:2022, A.8.8) y su ausencia es un hallazgo de diseño.

Paso E — Consolidar en una matriz de control única

Cuatro reportes en cuatro formatos distintos no son un papel de trabajo. Se consolidan:

```
# 30_papeles_trabajo/PT04_matriz_control.py
import pandas as pd, json, re, glob

filas = []

# --- Lynis ---
for l in open("../20_evidencia/E04_config/lynis-report.dat"):
    if l.startswith("warning["]=) or l.startswith("suggestion["]=):
        tipo = "warning" if l.startswith("warning") else "suggestion"
        filas.append(dict(herramienta="Lynis", severidad="Alta" if tipo=="warning"
else "Media",
                        hallazgo=l.split("=",1)[1].strip()[:160]))

# --- Docker Bench ---
for l in open("../20_evidencia/E04_config/docker-bench.log"):
    if l.startswith("[WARN]"):
        filas.append(dict(herramienta="Docker Bench", severidad="Alta",
                        hallazgo=l.replace("[WARN]", "").strip()[:160]))

# --- Trivy imágenes ---
for f in glob.glob("../20_evidencia/E04_config/trivy_*.json"):
    d = json.load(open(f))
    for res in d.get("Results", []):
        for v in res.get("Vulnerabilities", []) or []:
            filas.append(dict(herramienta="Trivy",
severidad=v["Severity"].capitalize(),
                        hallazgo=f'{v["VulnerabilityID"]} en {v["PkgName"]}
{v.get("InstalledVersion","")}''))

m = pd.DataFrame(filas)

# --- MAPEO AL CRITERIO • sin esto la salida es ruido, no auditoría ---
MAPEO = [
    (r"root|privileg|capab", "A.8.2 Privileged access rights",
"DSS05.04"),
```



```

(r"password|credential|secret|auth", "A.5.17 Authentication information",
"DSS05.04"),
(r"CVE-|vulnerab|outdated|version", "A.8.8 Management of technical
vulnerabilities", "DSS05.07"),
(r"log|audit|journal", "A.8.15 Logging", "DSS01.03"),
(r"tls|ssl|cipher|encrypt|certificate", "A.8.24 Use of cryptography",
"DSS05.03"),
(r"firewall|port|network|expose", "A.8.20 Networks security",
"DSS05.02"),
(r"config|default|hardening", "A.8.9 Configuration management",
"BAI10.02"),
(r"backup|restore", "A.8.13 Information backup",
"DSS04.07"),
]
def clasifica(t):
    for pat, iso, cobit in MAPEO:
        if re.search(pat, t, re.I):
            return pd.Series([iso, cobit])
    return pd.Series(["Sin clasificar", "Sin clasificar"])

m[["control_iso27001", "objetivo_cobit"]] = m["hallazgo"].apply(clasifica)
m.to_csv("../40_hallazgos/PT04_matriz_control.csv", index=False)

print("Hallazgos por herramienta:\n",
m.groupby(["herramienta", "severidad"]).size().to_string())
print("\nHallazgos por control ISO:\n",
m["control_iso27001"].value_counts().to_string())
print(f"\nSin clasificar: {(m.control_iso27001=='Sin clasificar').sum()} "
f"({(m.control_iso27001=='Sin clasificar').mean():.0%}) – revisar manualmente")

```

Cierre de la sesión (10 min). Cada equipo elige el control con más hallazgos concentrados y responde en el papel de trabajo. ¿Esto es una **deficiencia de diseño** (el control no existe) o de **eficacia operativa** (el control existe pero no se ejecutó)? La respuesta determina la recomendación. En el primer caso se diseña el control; en el segundo se corrige el proceso que lo dejó de ejecutar.

```

sha256sum 20_evidencia/E04_config/* 40_hallazgos/PT04_matriz_control.csv >>
20_evidencia/SHA256SUMS_E04.txt
git add . && git commit -m "E04: auditoria de configuracion segura y matriz de control
consolidada"

```

Paso F — Validar y corregir (25 min)

El resultado no vale por estar hecho, sino por resistir una comprobación. Se ejecutan estas tres y **se corrige lo que falle antes de cerrar la sesión**.

1. Contar el porcentaje de hallazgos «Sin clasificar» en la matriz y comprobar que está por debajo del 20 %.
2. Verificar que cada regla fallida de severidad alta lleva su identificador XCCDF, no solo su descripción.



3. Comprobar que el control analizado como deficiencia de **diseño** lo es de verdad. El control no existe, frente a existe pero no opera.

Lo que no se pueda corregir hoy se anota en la sección **Problemas y mejoras** de la evidencia, con lo que faltó y por qué. Un resultado parcial documentado con honestidad vale más que uno declarado sin prueba.

Paso G — Registrar la evidencia y cerrar (15 min)

Se versiona lo producido, se anota la URL de cada resultado y se responde en dos frases la pregunta de transferencia — **qué riesgo correría una organización real si esto se hiciera mal**.

3. Resultados

3.2 Lista de comprobación - Item 1 (Lynis):

- **Herramienta:** Lynis v3.0.7
- **Índice obtenido:** Hardening index = 57
- **Tres advertencias/sugerencias analizadas:**
 1. ACCT-9628 (Habilitar auditd para recolectar información de auditoría) $\rightarrow$ Control ISO/IEC 27001:2022 **A.8.15 (Logging / Registros)**.
 2. FINT-4350 (Instalar herramienta de integridad de archivos como AIDE/Tripwire) $\rightarrow$ Control ISO/IEC 27001:2022 **A.8.9 (Gestión de la configuración)**.
 3. HRDN-7230 (Instalar escáner de malware periódico como rkhunter o chkrootkit) $\rightarrow$ Control ISO/IEC 27001:2022 **A.8.7 (Protección contra malware)**.
- **Evidencia:** Ver **Anexo A** para la captura completa.
- **URL en GitHub:** [https://github.com/elvisMamaniValdivia/auditoria-si084/blob/taller-04/20_evidencia/E04_config/lynis-report.dat] (https://github.com/elvisMamaniValdivia/auditoria-si084/blob/taller-04/20_evidencia/E04_config/lynis-report.dat)

Evidencia obligatoria en GitHub. Todo resultado de este taller se versiona en el repositorio del equipo. El informe **no consigna capturas sueltas**. Consigna la **URL** del artefacto en GitHub. Una captura no permite verificar autoría, fecha ni contenido; un enlace sí.



Qué se entrega	Dónde vive	Qué se escribe en el informe
Código y archivos de configuración	Rama del taller, fusionada a develop vía Pull Request	URL del Pull Request
Documentos y matrices	docs/, en formato de texto versionable	URL del archivo en la rama
Capturas y videos que el taller exija	docs/evidencias/S04/	URL del archivo
Salida de comandos	docs/evidencias/S04/salidas/*.txt	URL del archivo

3.1. Los tres resultados que se califican

Son los que la rúbrica evalúa. El resto de la lista tiene que existir, pero no se califica fila por fila.

Resultado	Qué demuestra	Dónde está
La matriz de control única	Los hallazgos de las cuatro herramientas mapeados a ISO/IEC 27001 y COBIT 2019	PT04_matriz_control.csv
La clasificación completada	Menos del 20 % sin clasificar, con el resto resuelto a mano	Salida del script
Diseño frente a eficacia operativa	Un control analizado en ambos planos, con su justificación	Papel de trabajo

4. 3.2. Lista de comprobación del taller

Todo esto debe existir al cerrar la sesión.

#	Resultado esperado	Verificación
1	lynis-report.dat con hardening_index extraído y las 3 advertencias de mayor severidad analizadas	Papel de trabajo
2	oscap-reporte.html con el perfil evaluado y las 5 reglas fallidas de severidad alta con su identificador XCCDF	Reporte HTML



#	Resultado esperado	Verificación
3	docker-bench.log con el conteo de [WARN] y el análisis de las secciones 4 y 5	Log y papel de trabajo
4	Reportes de Trivy. Imágenes, IaC, secretos y un SBOM en CycloneDX	Archivos generados
5	PT04_matriz_control.csv con todos los hallazgos mapeados a ISO/IEC 27001 y COBIT 2019	Contenido del CSV
6	Menos del 20 % de hallazgos «Sin clasificar» , con los restantes clasificados manualmente	Salida del script
7	Un control analizado explícitamente como deficiencia de diseño vs. de eficacia operativa , con justificación	Papel de trabajo
8	Hashes en la cadena de custodia y <i>commit</i> en Git	SHA256SUMS_E04.txt

5. Rúbrica procedimental (20 puntos)

Se aplica sobre el informe entregado y la evidencia enlazada en el repositorio. **Cada criterio se califica de forma independiente.**

Criterio	4 — Logrado	2 — En proceso	0 — Insuficiente
Auditoría del sistema anfitrión con Lynis	Completo y correcto, con la evidencia que lo respalda	Completo con errores menores, o correcto pero sin toda la evidencia	Incompleto, o entregado sin ejecutar
Vulnerabilidades, IaC y secretos con Trivy	Completo y correcto, con la evidencia que lo respalda	Completo con errores menores, o correcto pero sin toda la evidencia	Incompleto, o entregado sin ejecutar
Evidencia verificable en el repositorio	Cada resultado tiene su URL sobre la etiqueta taller-NN, y el enlace abre lo que dice	La mayoría tiene URL; alguna evidencia es una captura suelta	Se declaran resultados sin enlace, o el enlace no corresponde



Criterio	4 — Logrado	2 — En proceso	0 — Insuficiente
Trazabilidad de la evidencia	Todo hallazgo o dato se rastrea hasta el archivo, registro y fecha que lo sustenta	Rastreable en su mayoría; algún dato sin origen	Se afirman hechos sin poder ubicarlos en la evidencia
La evidencia entregada	Las secciones de la plantilla completas; los papeles de trabajo quedan archivados y referenciados	Secciones completas con papeles de trabajo incompletos	Faltan secciones o no hay papeles de trabajo

4. Conclusiones

Mínimo tres. Una conclusión no resume lo que hiciste. Dice lo que aprendiste y se sostiene en la evidencia de la sección 3.

5. Cuestionario

Copia cada pregunta de la guía de la semana y respóndela debajo.

6. Referencias bibliográficas

- ISO/IEC 27001:2022, Anexo A, temas A.7 (Físicos) y A.8 (Tecnológicos). <https://www.iso.org/standard/27001>
- ISO/IEC 27002:2022. *Information security controls*. <https://www.iso.org/standard/75652.html>
- ISACA. (2018). *COBIT 2019 Framework: Governance and Management Objectives*, dominios BAI y DSS. <https://www.isaca.org/resources/cobit>
- COSO (Committee of Sponsoring Organizations of the Treadway Commission). (2013). *Internal Control — Integrated Framework*. <https://www.coso.org/guidance-on-ic>
- U.S. Congress. (2002). *Sarbanes-Oxley Act of 2002*, Public Law 107-204, secciones 302 y 404. <https://www.govinfo.gov/app/details/PLAW-107publ204>
- Superintendencia de Banca, Seguros y AFP. *Resolución SBS N.º 504-2021, Reglamento para la Gestión de la Seguridad de la Información y la Ciberseguridad*. <https://www.sbs.gob.pe/>
- Ley 28716, Ley de Control Interno de las Entidades del Estado. <https://www.gob.pe/contraloria>
- Center for Internet Security. *CIS Benchmarks*. <https://www.cisecurity.org/cis-benchmarks>
- CISOfy. *Lynis — Security auditing tool*. <https://cisofy.com/lynis/>
- OpenSCAP Project. <https://www.open-scap.org/> · ComplianceAsCode. <https://github.com/ComplianceAsCode/content>
- Aqua Security. *Trivy Documentation*. <https://trivy.dev/>



- ISO/IEC 25010. *Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. <https://www.iso.org/standard/78176.html>
- Piattini Velthuis, M., Del Peso Navarro, E. y Del Peso Ruiz, M. (2009). *Auditoría de tecnologías y sistemas de información* (6.ª ed.). Alfaomega / Ra-Ma.

7. Anexos

Capturas completas, archivos de configuración y salidas extensas. Cada anexo lleva su letra y su título, y se menciona en el cuerpo del informe.

Anexo A — Salida de ejecución y Hardening Index de Lynis

```
Lynis security scan details:
Hardening index : 57 [##### ]
Tests performed : 228
Plugins enabled : 1
```

Anexo B — Reporte completo de cumplimiento con OpenSCAP (anexo_A_oscapede.html / oscapede.html)

Evaluation Characteristics

Evaluation target	24b627d3f50e
Benchmark URL	#scap_org.open-scap_comp_ssg-rhel9-xccdf.xml
Benchmark ID	xccdf_org.ssgproject.content_benchmark_RHEL-9
Benchmark version	0.1.81
Profile ID	xccdf_org.ssgproject.content_profile_cis_server_l1
Started at	2026-09-09T21:37:11+00:00
Finished at	2026-09-09T21:37:11+00:00
Performed by	root
Test system	cpe:/a:redhat:openscap:1.3.14

CPE Platforms

- cpe:/o:rocky:rocky:9
- cpe:/o:redhat:enterprise_linux:9

Addresses

- IPv4 127.0.0.1
- IPv4 172.17.0.2
- IPv6 0:0:0:0:0:0:1
- MAC 00:00:00:00:00:00
- MAC 8A:91:1D:C6:ED:B0